

ENTERPRISE ARCHITECTURE DIVISION

RESTful API Design & Architecture Standards

Hospital Management System (HMS) Core Blueprint

Author:	Senior Backend Architect
Target Stack:	Spring Boot & MongoDB Atlas
Version:	1.0.0 (Production Ready)
Date:	June 12, 2026
Status:	Approved Blueprint

1. Architectural Overview & Policy

This document outlines the mandatory API standards for the next-generation Hospital Management System (HMS). As an enterprise system handling Critical Health Information (CHI) and Personal Health Information (PHI), all APIs must prioritize consistency, robust validation, structural clarity, deterministic error handling, and strict cryptographic security. The underlying technology platform utilizes **Spring Boot 3.x** combined with **MongoDB** as the transactional primary datastore.

2. API Versioning Policy

To prevent breaking changes and allow smooth evolution of business logic, all exposed endpoints must embed versioning directly into the uniform resource identifier (URI).

- **Strategy:** URI Path Versioning.
- **Format:** lowercase `v` followed by an integer counter (e.g., `v1`, `v2`). Minor and patch updates must not change the URI prefix.
- **Base URL Pattern:** `https://api.hms-core.com/api/v1/{resource}`

Architectural Note: Never expose raw datastore collection structures or directly map database documents to the client layout without an API version abstraction layer (DTO pattern).

3. REST API Naming Conventions

Endpoints must be designed around clean, logical resources instead of actions or database methods. Adhere to the following conventions strictly:

- **Nouns over Verbs:** Use pluralized nouns representing resources (e.g., `/patients` instead of `/getPatient`).
- **Casing:** Use strict `kebab-case` for multi-word paths (e.g., `/medical-records`).
- **Lowercase Hierarchy:** All characters must be lowercase. Do not use trailing slashes.

Standard HTTP Method Mapping

Method	URI Example	Semantic Interpretation
GET	/api/v1/patients	Retrieve a paginated, filtered list of patients.
GET	/api/v1/patients/{id}	Retrieve specific patient metadata by their unique identifier.
POST	/api/v1/patients	Create a new patient file. Returns HTTP 201 Created.
PUT	/api/v1/patients/{id}	Replace/Update the entire state of an existing patient document.
PATCH	/api/v1/patients/{id}	Partially modify an existing record (e.g., change room assignment).
DELETE	/api/v1/patients/{id}	Perform a logical soft-delete of the patient record.

4. Request Format Specification

All mutating operations (**POST** , **PUT** , **PATCH**) must exchange payload data using JSON. Headers must enforce encoding properties.

Mandatory HTTP Headers

- Content-Type:** `application/json; charset=UTF-8`
- Accept:** `application/json`
- X-Correlation-ID:** `<UUID>` (Required for telemetry and distributed logging trace propagation)

Example: Patient Admission Request (POST)

```
POST /api/v1/appointments HTTP/1.1
Host: api.hms-core.com
Content-Type: application/json
X-Correlation-ID: a8b3d942-51c0-478a-bb0a-20d753d0434a

{
  "patientId": "64f1bc20e4b0c2a5d89f81a1",
  "doctorId": "64f1bc32e4b0c2a5d89f81a5",
  "appointmentDateTime": "2026-06-15T10:30:00Z",
  "department": "CARDIOLOGY",
  "symptoms": [
    "Acute chest pain",
    "Shortness of breath"
  ]
}
```

5. Response Format Specification

To preserve consistency across all frontend consumers (Web Portal, Mobile Apps), the HMS ecosystem mandates a standardized envelope format for all data payloads.

Standard Success Response Structure

```
{
  "success": true,
  "timestamp": "2026-06-12T16:12:34Z",
  "correlationId": "a8b3d942-51c0-478a-bb0a-20d753d0434a",
  "data": {
    "appointmentId": "64f1bd99e4b0c2a5d89f81b9",
    "status": "CONFIRMED",
    "queueNumber": 12
  },
  "meta": null
}
```

Standard Paginated Response Structure (Collection Arrays)

```
{
  "success": true,
  "timestamp": "2026-06-12T16:15:00Z",
  "correlationId": "b11cd831-29a4-419b-a312-32b123d919ab",
  "data": [
    { "id": "64f1bc20e4b0c2a5d89f81a1", "name": "John Doe" }
  ],
  "meta": {
    "pageNumber": 0,
    "pageSize": 20,
    "totalElements": 145,
    "totalPages": 8,
    "isLast": false
  }
}
```

6. Robust Error Handling Schema

When an execution anomaly occurs, the application must never spill internal raw server components or Java stack traces to the network layer. Error responses must present explicit debugging structures under a standard layout.

Global Error Contract

Field Path	Type	Semantic Definition
success	Boolean	Always hardcoded to false during error states.
error.code	String	Application-specific error sub-code mapped to internal rule books.
error.message	String	High-level human-readable clarification text.
error.details	Array (Objects)	Sub-structural breakdown listing individual field-level validation errors.

Example: Field Validation Breakdown (HTTP 400 Bad Request)

```
{
  "success": false,
  "timestamp": "2026-06-12T16:18:22Z",
  "correlationId": "c88f1211-1a22-44bb-99cc-432a11b8bca1",
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "The payload values failed business constraint validation checking.",
    "details": [
      {
        "field": "appointmentDateTime",
        "rejectedValue": "2020-01-01T00:00:00Z",
        "issue": "Appointment date must be scheduled in a future temporal window."
      }
    ]
  }
}
```

7. Authentication Strategy (Stateless JWT)

The HMS architecture uses stateless JSON Web Token (JWT) signatures executed via an asymmetric key paradigm (RS256) or a secure symmetric hash system (HMAC-SHA512).

- Tokens must be provided in the HTTP Authorization header using the Bearer scheme.
- **Payload Structure:** The payload section must specify standard claims, client attributes, security fingerprints, and access scopes.

Structure of Decoded Enterprise JWT Payload

```
{
  "iss": "https://auth.hms-core.com",
  "sub": "usr_992182741",
  "exp": 1781280000,
  "iat": 1781276400,
  "name": "Dr. Sarah Jenkins",
  "email": "s.jenkins@hms-hospital.org",
  "tenantId": "TENANT_EAST_CAMPUS",
  "roles": [
    "DOCTOR",
    "MEDICAL_STAFF"
  ],
  "permissions": [
    "PATIENT_RECORD_READ",
    "PATIENT_RECORD_WRITE",
    "PRESCRIPTION_CREATE"
  ]
}
```

8. Role-Based Authorization Model (RBAC)

Endpoints enforce a strict, zero-trust Role-Based Access Control configuration matrix. Roles dictate coarse structural constraints, while fine-grained permissions govern logical application filtering inside the Spring Security framework.

Access Control Permission Matrix

System Module Path	ADMIN	DOCTOR	NURSE	RECEPTIONIST
/api/v1/billing/**	Full Access	No Access	No Access	Read/Write
/api/v1/patients/**	Full Access	Full Access	Full Access	Read/Write (No Clinical Data)
/api/v1/medical-records/**	Full Access	Full Access	Read/Write	No Access
/api/v1/staff/**	Full Access	No Access	No Access	No Access

Security Directive: Annotate standard Java controller class endpoints explicitly using Spring Security Method Security pre-authorizations: `@PreAuthorize("hasRole('DOCTOR')")` .

9. Canonical Spring Boot Repository Architecture

To ensure clean separation of concerns and maintainability across separate engineering teams, the codebase adopts a domain-driven architectural packaging approach combined with functional layering.

```

com.hospital.management.core
├── HmsCoreApplication.java           # System Main Application Bootstrap
├── config                           # Enterprise Configuration Classes
│   ├── MongoConfig.java             # MongoDB Connection, Auditing & Codec Mappings
│   ├── SecurityConfig.java          # Spring Security, JWT Filter Hooks, CORS Rules
│   └── OpenApiConfig.java           # Swagger/OpenAPI v3 Specification Configuration
├── security                         # Encryption, Context & Authentication Layer
│   ├── JwtAuthenticationEntryPoint.java # Unauthorized Access Exception Overrides
│   ├── JwtAuthenticationFilter.java   # Interceptor Extracting & Validating Bearer String
│   └── TokenProvider.java             # Crypto Evaluation, Token Verification & Issuance
├── common                           # Cross-Cutting Shared Base Logic
│   ├── exception                    # Global System Exception Model Classes
│   │   ├── BusinessException.java
│   │   └── ResourceNotFoundException.java
│   ├── dto                          # Global Generic Envelope API Wrappers
│   │   ├── ApiResponse.java          # Standard JSON Data Envelope
│   │   └── ErrorResponse.java        # Error Payload Data Structures
│   └── advice
│       └── GlobalExceptionHandler.java # @ControllerAdvice Mapping Internal Errors to HTTP C
├── domain                           # Bound Context Modules (Domain Components)
│   ├── patient                      # Patient Management Context Sub-domain
│   │   ├── collection               # MongoDB Persistent Collection Documents
│   │   │   └── Patient.java          # Annotated with @Document(collection = "patients")
│   │   ├── repository               # MongoDB Driver Spring Data Data-Access Interfaces
│   │   │   └── PatientRepository.java # Extends MongoRepository<Patient, String>
│   │   ├── dto                      # Specialized Layer-to-Layer Request/Response Payload
│   │   │   ├── PatientCreatedto.java
│   │   │   └── PatientResponseDto.java
│   │   ├── service                  # Business Logic Definitions
│   │   │   ├── PatientService.java   # Interface
│   │   │   └── PatientServiceImpl.java # Transactional Implementation
│   │   └── controller               # Presentation Endpoint Interfaces
│   │       └── PatientController.java # Exposes Base URLs mapping HTTP methods
│   ├── appointment                  # Appointment Scheduling Context Module
│   │   ├── collection
│   │   │   └── Appointment.java
│   │   ├── repository
│   │   │   └── AppointmentRepository.java
│   │   ├── service
│   │   │   └── AppointmentServiceImpl.java
│   │   └── controller
│   │       └── AppointmentController.java
│   └── medicalrecord                # Confidential Clinical Records Context Module
│       ├── collection
│       │   └── MedicalRecord.java
│       └── repository
│           └── MedicalRecordRepository.java

```


10. Datastore Persistence Blueprint (MongoDB)

Because the primary transactional repository is MongoDB, developers must structure records following standard denormalization conventions optimized for document databases.

- **Identifiers:** All collections must declare their primary key using a 24-character hexadecimal string representing a MongoDB `ObjectId`, encapsulated as a String in Java.
- **Indexing:** Highly queried lookup variables must declare indexing tags explicitly inside the Java layer using the `@Indexed` annotation. Compound parameters (e.g., matching a patient identifier combined with time sequences) require a declarative cluster setup.
- **Auditing:** Enable Spring Data auditing frameworks (`@CreatedDate` , `@LastModifiedDate` , `@CreatedBy` , `@LastModifiedBy`) on all collections to fulfill tracking rules.